



Informatics II

Exercise 1

Grgic Anto

Schedule (Corrected)

Date	Topic
27. Feb	C environment
06. Mar	Ex1(Introduction to C, basic sorting)
13. Mar	Ex2 (Recursion)
20. Mar	Ex3 (Complexity, Correctness)
27. Mar	Ex4 (Divide and Conquer, Recurrence)
03. Apr	NO LAB (Please attend another class) – Spring break
17. Apr	Ex6 (Pointers, Linked Lists)
24. Apr	Ex7(ADTs, Stacks, Queues)
01. May	NO LAB (Please attend another class) – Labor day
08. May	Ex9 (Hash Tables)
15. May	Ex10 (Dynamic Programming)
22. May	Ex11 (Graphs)
29. May	Recap
04. Jun	Exam at Messe 16:00-19:00

From last week:

```
1  #include <stdio.h>
2  #define N 5
3
4  int counter;          //ASSIGN VALUE
5
6  void greet();          //DECLARED THE BLUEPRINT BEFORE
7
8  int add(int a, int b);
9
10 int main(){            //MAIN FUNCTION
11     counter = add(2, 3);
12
13     printf("%d\n", counter);
14
15     greet();
16
17     return 0;
18 }
19
20 int add(int a, int b){   //DEFINE FUNCTION
21     return a + b;
22 }
23
24 void greet(){
25     printf("Hello!\n");
26 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\antog\Desktop\Scuola\Semestre_4\INFO2_TUTORAT\Sessions\Session1> gcc example.c -o example
PS C:\Users\antog\Desktop\Scuola\Semestre_4\INFO2_TUTORAT\Sessions\Session1> ./example
5
Hello!
```

```
1  #include <stdio.h>
2  #define N 5
3
4  int counter;          //ASSIGN VALUE
5
6  int main(){            //MAIN FUNCTION
7     counter = add(2, 3);
8
9     printf("%d\n", counter);
10
11     greet();
12
13     return 0;
14 }
15
16 int add(int a, int b){   //DEFINE FUNCTION
17     return a + b;
18 }
19
20 void greet(){
21     printf("Hello!\n");
22 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\antog\Desktop\Scuola\Semestre_4\INFO2_TUTORAT\Sessions\Session1> gcc example.c -o example

```
example.c: In function 'main':
example.c:7:15: warning: implicit declaration of function 'add' [-Wimplicit-function-declaration]
7 |     counter = add(2, 3);
  |               ^~~
example.c:11:5: warning: implicit declaration of function 'greet' [-Wimplicit-function-declaration]
11 |     greet();
    |     ^~~~~~
example.c: At top level:
example.c:20:6: warning: conflicting types for 'greet'; have 'void()'
20 | void greet(){
    | ^~~~~~
example.c:11:5: note: previous implicit declaration of 'greet' with type 'void()'
11 |     greet();
    |     ^~~~~~
```

Either declare the function before the main, or define it completely before!!

From last week:

```
char str[] = "Hello";
```

```
char str[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Equivalent, but the first option is easier to write

More c syntax

There are two standard main signatures

```
int main(void)
int main(int argc, char **argv)
```

```
./program 1 2 3 4
```

argc = 5
argv[0] = “./program”
argv[1] = “1”
.
.
.
argv[4] = “4”

argc = the length of the input.

**argv = a pointer to pointer of chars, or an array of pointer to chars, or an array of strings.

```
sscanf(argv[1], "%d", &m[0][0]);
sscanf( "1", "%d", &m[0][0]);
```

***Note that argv[i] are not integer but chars, so if you want to use them as numbers you need to convert them to integers.**

We will come back to this example once pointer are seen in class!

Bubble sort

Algo: BubbleSort(A)

```
for i = n to 2 do
  for j = 2 to i do
    if A[j] < A[j-1] then
      tmp = A[j];
      A[j] = A[j-1];
      A[j-1] = tmp;
```

Number of comparisons:

$$C = \sum_{i=2}^n (i-1) = \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} = \frac{n^2 - n}{2}$$

Number of moves:

$$M_{min} = 0$$

$$M_{max} = \sum_{i=2}^n 3(i-1) = \frac{3n(n-1)}{2} = \frac{3n^2 - 3n}{2}$$

Selection sort

Algo: SelectionSort(A)

```
for i = 1 to n-1 do  
  k = i;  
  for j = i+1 to n do  
    if A[j] < A[k] then k=j;  
  exchange A[i] and A[k];
```

Number of comparisons:

$$C = \sum_{i=2}^n (i-1) = \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} = \frac{n^2 - n}{2}$$

Number of moves:

$$M = \sum_{i=1}^{n-1} 3 = 3(n-1) = 3n - 3$$

Insertion sort

Algo: InsertionSort(A)

for $i = 2$ **to** n **do**

$j = i-1$;

$tmp = A[i]$;

while $j \geq 1 \wedge tmp < A[j]$

do

$A[j+1] = A[j]$;

$j = j-1$;

$A[j+1] = tmp$;

Number of comparisons:

$$C_{min} = \sum_{i=2}^n 1 = n - 1$$

$$C_{max} = \sum_{i=2}^n (i - 1) = \frac{n^2 - n}{2}$$

Number of moves:

$$M_{min} = \sum_{i=2}^n 2 = 2(n - 1) = 2n - 2$$

$$M_{max} = \sum_{i=2}^n (i + 1) = \frac{n^2 + 3n - 4}{2}$$

Recursion

```
int factorial(int n) {  
  
    if (n == 0) return 1; // base case  
  
    return n * factorial(n - 1); // body + recursive call  
}
```

↓ CALLING

factorial (4) → calls factorial(3) CALL

factorial (3) → calls factorial(2) CALL

factorial (2) → calls factorial(1) CALL

factorial (1) → calls factorial(0) CALL

factorial (0) → 1 BASE CASE

↑ RETURNING

factorial (1) → $1 \times 1 = 1$ RETURN

factorial (2) → $2 \times 1 = 2$ RETURN

factorial (3) → $3 \times 2 = 6$ RETURN

factorial (4) → $4 \times 6 = 24$ FINAL

Base case: A simple value is returned. The calling “loop” stops and the returning “loop” starts.

Body: Some kind of operation get applied (Often to the recursive call’s returning value).

Recursive call: The function calls itself with a parameter that depends on the parameter of the current iteration.

Task 1.

```
#include <stdio.h>
int main(){
    int a[]={15,5,1,4,2,8,7},n=7,t;
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-1-i;j++)
            if(a[j]>a[j+1]){t=a[j];a[j]=a[j+1];a[j+1]=t;}
        for(int k=0;k<n;k++)printf("%d_",a[k]);
        printf("\n");
    }
}
```

-How would you change the algorithm if:

- You wanted to start from the right side?
- You wanted to order it in descending order?
- you wanted to start from the right side and order it in descending order?

Task 2.

Algo: XSort(A)

$l = 1; r = n;$

do

for $j = r$ **to** $l + 1$ **do**

if $A[j] < A[j-1]$ **then**

 exchange $A[j]$ and $A[j-1];$

$m = j;$

$l = m;$

for $j = l$ **to** $r - 1$ **do**

if $A[j] > A[j+1]$ **then**

 exchange $A[j]$ and $A[j+1];$

$m = j;$

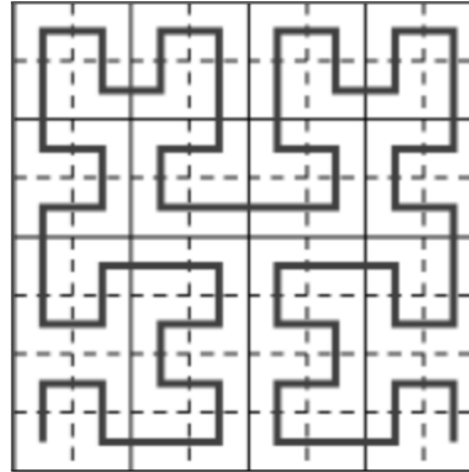
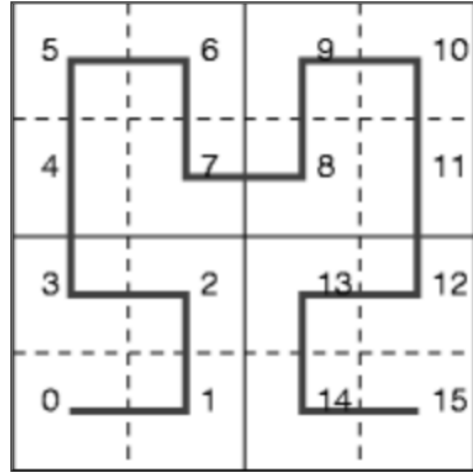
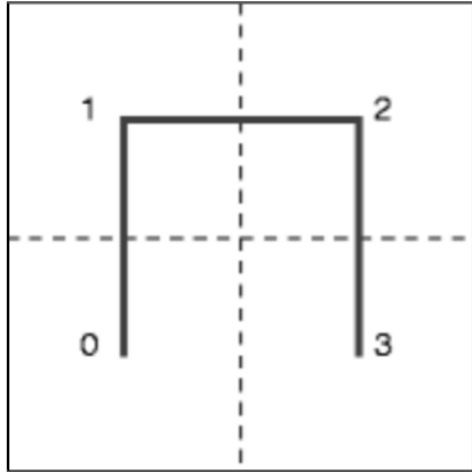
$r = m;$

while $l < r;$

$A = (4, 8, 8, 7, 7, 3, 5, 0, 5),$

- Write down the values you know at the beginning.
- Write down the values after each loop.

Task 3.



- Start simple and abstract.
- Try to find a pattern
- Add specific values later

$$S_n \equiv \sum_{k=0}^n r^k = \frac{1 - r^{n+1}}{1 - r}.$$

Task 4.

- Simple task to get you familiar with c

```
int main(int argc, char **argv) {  
    int i, j, k;  
  
    sscanf(argv[1], "%d", &m[0][0]);    //This has to do with pointers,  
    sscanf(argv[2], "%d", &m[0][1]);    //we'll look it in depth in class no.6  
    sscanf(argv[3], "%d", &m[1][0]);  
    sscanf(argv[4], "%d", &m[1][1]);  
}
```

```
printf("%3d", m[i][j]);    //%3d just means make it at least 3 char long  
                           //ex: m[1][j] = 3 -> "  3"  
                           //ex: m[i][j] = 30 -> " 30"
```

Task 5.

- 5.1 Develop and implement an algorithm `pairSum` that solves the problem in C.
- 5.2 Assume array A is sorted in non-decreasing order. Develop an algorithm `pairSumSorted` that exploits the sortedness and solves the problem faster than algorithm `pairSum`.
- 5.3 Assuming the input array A is sorted in non-decreasing order, determine the worst-case time complexity of both `pairSum` and `pairSumSorted`. In addition, quantify the difference in their worst-case wall-clock running times when the sorted array A contains 500,000 concrete numbers.

Thank you for your attention.

**If you have any question write in the Olat forum or
come up to me.**